

Article

Development of a web-based FAQ chatbot for academic information services: A case study of Bayero University

Kano

Habeeba Adamu Kakudi¹, Hamzat Abdulroqib², and Zainab Magaji Musa^{3,*}

¹ Department of Computer Science, Faculty of Computing, Bayero University Kano, Nigeria; hakakudi.cs@buk.edu.ng

² Department of Computer Science, Faculty of Computing, Bayero University Kano, Nigeria; ha2000520.com@buk.edu.ng

³ Department of Computer Science, Faculty of Computing, Bayero University Kano, Nigeria; zmm2100015.pcs@buk.edu.ng

* Correspondence: zmm2100015.pcs@buk.edu.ng

Abstract: Access to accurate academic information within universities is often slow and inefficient due to reliance on manual communication channels such as notice boards, emails, and in-person inquiries. These limitations create delays, inconsistencies, and increased administrative workload. This study presents the design and implementation of a web-based Frequently Asked Questions (FAQ) chatbot for the Faculty of Computing at Bayero University Kano. The system provides real-time responses to student queries through an automated conversational interface. The chatbot architecture integrates a React frontend with a Django REST Framework backend and a structured FAQ database managed through an administrative dashboard. User queries are processed using keyword-matching techniques to retrieve the most relevant stored responses, while unmatched queries trigger fallback responses. The system also supports user authentication and FAQ management functionalities. The developed solution improves accessibility to faculty information, reduces repetitive administrative tasks, and enables continuous 24/7 support for students. Evaluation through functional and integration testing confirms that the chatbot provides accurate and timely responses under typical usage conditions. The system demonstrates the feasibility of deploying lightweight academic chatbots in resource-constrained institutional environments. It provides a scalable foundation for future integration of advanced natural language processing capabilities across university departments.

Keywords: chatbot, FAQ automation, academic information systems, Django REST Framework, React

Received: 24 February 2026; Revised: 12 April 2026; Accepted: 24 April 2026; Published Online: 30 April 2026



Copyright: ©2026 the Author(s). Published by JSSCI. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Journal Abbreviation: J. Stat. Sci. Comput. Intell.

1. Introduction

Access to accurate information and efficient communication are critical determinants of institutional

effectiveness and academic success in higher education environments [1]. Students in universities such as Bayero University Kano (BUK) frequently require reliable and up-to-date information regarding departmental procedures, examinations, course registration, and academic policies. However, conventional information dissemination methods, including notice boards, printed memoranda, emails, and in-person inquiries, are often inefficient, delayed, and inaccessible to students outside campus or administrative hours [2]. These limitations create communication gaps, inconsistent information delivery, and increased administrative workload for faculty staff.

The rapid evolution of digital technologies is transforming contemporary learning environments and driving demand for automated systems that improve accessibility and streamline institutional communication [3]. Conversational agents and academic chatbots have emerged as effective tools for delivering instant, consistent, and scalable information services within universities. Such systems leverage structured knowledge bases and natural language processing (NLP) techniques to provide context-aware responses, thereby reducing repetitive administrative tasks and improving user experience [4].

Despite these advances, the Faculty of Computing at Bayero University Kano continues to rely predominantly on manual information channels, resulting in delayed responses to frequently asked student queries, repetitive staff workload, limited after-hours accessibility, and potential inconsistencies in disseminated information. These challenges highlight the need for an automated academic information system capable of delivering timely and reliable responses through a conversational interface.

This study addresses these challenges by designing and implementing an intelligent web-based FAQ chatbot for the Faculty of Computing, Bayero University Kano. The proposed system integrates a structured FAQ database with lightweight NLP-based query processing and modern web technologies, specifically a React frontend and Django REST Framework backend. The chatbot provides real-time responses to frequently asked questions, supports fallback mechanisms for unmatched queries, and enables secure administrative management of knowledge content. The developed system aims to enhance communication efficiency, improve accessibility to faculty information, and reduce administrative workload by automating repetitive inquiries. Furthermore, the platform is designed as a scalable foundation for future integration of advanced NLP models and intelligent conversational capabilities across university departments. By demonstrating the feasibility of deploying lightweight academic chatbots in resource-constrained institutional settings, this study contributes to the growing adoption of intelligent information systems in higher education.

2. Related Works

The application of chatbots in higher education has been widely explored as a method to improve information accessibility and reduce administrative workload. Ranoliya et al. [6] developed an AIML-based chatbot system for automating university FAQs, combining template-based responses via Artificial Intelligence Markup Language (AIML) with Latent Semantic Analysis (LSA) for service-based retrieval. Their hybrid approach provided faster response times and increased decision accuracy for common student and faculty queries. However, the system lacked user authentication, personalized chat histories, and robust fallback handling, limiting its ability to manage complex, multi-turn, or out-of-domain queries.

Clarizia et al. [7] proposed an education support system leveraging Natural Language Processing (NLP) combined with domain ontologies to provide accurate, domain-specific answers. Their ontology-driven approach improved retrieval accuracy over traditional rule-based systems. Despite this, building and maintaining complex ontologies proved resource-intensive, and the system lacked a comprehensive user authentication framework.

Lee et al. [8] evaluated the effectiveness of an FAQ chatbot in reducing administrative workload through NASA-

TLX and interaction log analysis. While the chatbot helped manage repetitive queries and influenced work patterns, the small-scale study across only two offices limited generalizability and did not deeply evaluate the robustness of the NLP engine.

Huddar et al. [9] developed “Dexter,” a college FAQ chatbot embedded in a university website to provide 24/7 support using NLP for intent recognition. The system successfully handled multiple concurrent users, reducing physical inquiries during high-traffic periods. However, the evaluation focused on prototype implementation, and long-term performance metrics or advanced fallback strategies were not extensively discussed.

Han and Lee [10] investigated FAQ chatbots in Massive Open Online Courses (MOOCs), comparing an NLP-based chatbot with traditional FAQ webpages. Findings indicated that learners sometimes faced higher interaction barriers with chatbots, and demographic factors significantly influenced user experience. The study was limited to a MOOC environment, limiting its applicability to conventional university settings.

Labadze et al. [11] conducted a systematic literature review of 67 studies on AI chatbots in education, noting their potential to reduce administrative workload and provide continuous support. Concerns about accuracy, data privacy, and ethical use remain, and most studies focused on short-term impacts rather than long-term pedagogical outcomes.

Bilquise et al. [12] examined student acceptance of an academic advising chatbot using TAM and UTAUT frameworks. The study of 207 students revealed that perceived ease of use and social influence strongly influenced adoption, while perceived usefulness was less significant. Limitations included geographic specificity and focus on advising rather than general faculty FAQs.

Mangotra et al. [13] explored deep-learning chatbots utilizing bidirectional LSTM and feed-forward neural networks, with basic administrative monitoring for FAQs. The platform improved retrieval accuracy over rule-based methods but lacked secure authentication and personalized user histories.

Zaky and Ratnasari [14] proposed a dual-mode academic chatbot combining rule-based responses with hybrid reasoning incorporating limited generative processing. Evaluation showed hybrid reasoning increased accuracy but required longer response times, highlighting a trade-off between computational efficiency and semantic flexibility. The system also lacked advanced admin update capabilities and authentication mechanisms.

3. Methodology

The system development followed an iterative Agile methodology, enabling incremental delivery of features and continuous testing at the end of each short development cycle [15]. Key workflow steps include: Requirement specification and functional analysis, Frontend development in React with interactive chatbot UI, Backend API development in Django REST Framework, integrating FAQ retrieval and fallback mechanisms, System integration and testing using functional and integration tests and User acceptance testing with a sample of students to evaluate response accuracy, interface usability, and overall system performance.

3.1. Description of Existing System

Currently, Bayero University Kano’s Faculty of Computing uses a manual information dissemination system. Students rely on physical notice boards, WhatsApp groups, emails, and in-person inquiries at administrative offices to access faculty information. This method is extremely time-consuming and inefficient. Students must physically visit campus to check notice boards, which may contain outdated or conflicting information. Administrative staff spend considerable time responding to repetitive queries, with approximately 70% of daily inquiries covering the same topics.

The existing system presents several challenges. Information posted on notice boards may take hours or days

to reach students; off-campus students and those on industrial attachments have limited access to timely information; different sources sometimes provide conflicting details, leading to confusion; and there is no search functionality for specific information. During peak periods such as registration and examinations, administrative offices experience queues of 15-20 students with wait times averaging 30-45 minutes. In summary, the existing system at the Faculty of Computing is laborious, inconsistent, and creates bottlenecks for both students and staff. The new FAQ Chatbot system aims to replace this with an online, real-time, 24/7 accessible solution.

3.2. Requirement Elicitation

To understand user needs, interviews with faculty administrative staff and students were conducted, as interviews are a widely accepted method for gathering system requirements from stakeholders [16].

1. Faculty Administrative Staff:

Needs/Goals: Reduce repetitive inquiry workload and improve efficiency. Staff want a system that can automatically answer frequently asked questions, allowing them to focus on complex issues requiring human judgment.

Challenges: Currently overwhelmed by 10-30 student inquiries daily during peak periods, with 70% being repetitive questions. Staff expressed frustration with the time spent answering the same questions repeatedly about registration, examinations, and results

2. Students:

Needs/Goals: Instant, 24/7 access to faculty information without physical campus visits. Students want a reliable, searchable system that provides consistent, accurate information.

Challenges: Survey results from 40 students showed that 78% had missed important information due to the current system, 66% were dissatisfied with information accessibility, and 89% indicated they would use a chatbot system if available. Off-campus students particularly struggle with timely access, often relying on friends or unofficial WhatsApp groups that sometimes provide conflicting information.

3.3. Functional Requirements

1. User Query Processing (Students/Staff): Users must be able to input queries in natural language text format through the chat interface and receive instant responses from the FAQ database.
2. FAQ Database Management (Admin): The admin must be able to create, update, and delete FAQ entries in the database, including question text, answer content, and category assignments, and also be able to delete students from the system
3. Keyword Matching Algorithm: The system must implement an FAQ search and keyword matching logic that retrieves the most relevant answer based on user queries.
4. Fallback Response Integration: When queries are not found in the FAQ database, the system must query a third-party API (Gemini API) to provide fallback responses.
6. User Authentication and Access Control: Admin users must log in with credentials to access the admin dashboard. Role-based permissions ensure only authorized staff can modify FAQ records; regular users have read-only access to the chat interface.
7. Conversation Management: Users can start a new conversation at any time, clearing chat history and beginning fresh interactions.

3.4. Non-Functional Requirements:

1. Performance: The system should respond to verification queries within 5 seconds under normal load

conditions. The system must handle at least 20 concurrent users without performance degradation.

2. Availability and Reliability: The chatbot portal should be available 24 hours a day, 7 days a week, with at least 95% uptime during the prototype phase. The system should handle errors gracefully without crashing, displaying user-friendly error messages.

3. Usability: The chatbot interface must be intuitive and user-friendly, requiring no training for basic usage. The system should provide clear instructions and helpful prompts to guide users. The interface should be responsive and adapt to different screen sizes (desktop, tablet, mobile).

4. Security: Admin passwords must be securely hashed and stored in the database. API keys for third-party services should be securely stored and not exposed in client-side code. Admin sessions should timeout after a period of inactivity to prevent unauthorized access

5. Scalability: The database design should support growth in FAQ entries without performance impact. The system architecture should allow for future integration of advanced natural language processing models.

6. Compatibility: The system should be compatible with major web browsers (Chrome, Firefox, Safari, and Edge) and function correctly on different operating systems (Windows, macOS, Linux, Android, and iOS).

This set of requirements forms the basis for the system design and will guide implementation in the following phases.

3.5. Use Case Diagram

The Faculty of Computing FAQ Chatbot system is designed to facilitate efficient and real-time information access for students while providing administrative staff with tools to manage the FAQ database [15]. The system primarily involves two actors: Students (End-Users) and Admin (Faculty Staff), each with specific interactions with the chatbot. The main use cases are illustrated in Figure 1 and described as follows:

Actors:

1. Student (End-User):

Students and faculty members interact with the chatbot to obtain information related to courses, registration, examinations, and other departmental operations. They rely on the chatbot for quick, accurate, and accessible answers without needing to visit administrative offices physically.

2. Admin:

Administrative staff manage the content and functionality of the chatbot, including updating FAQs, managing user accounts, and monitoring interactions to ensure accurate information delivery.

Use Cases:

1. Login:

Both students and admins authenticate themselves to access the system. Students are directed to the chatbot interface, while admins access the management dashboard. Authentication ensures secure access and personalized interaction.

2. Ask Question:

The primary function for students, allowing them to input queries into the chatbot interface. The system retrieves responses from the FAQ database using keyword matching or triggers fallback mechanisms for unmatched queries.

3. Start New Conversation:

Students can clear the current conversation history and initiate a new session, enabling them to ask unrelated or follow-up questions without confusion from previous interactions.

4. Manage FAQs:

Admins can add, edit, or delete FAQ entries to maintain up-to-date and relevant content. This use case

ensures that the chatbot remains accurate and responsive to evolving faculty information needs.

5. Manage Users:

Admins can delete or deactivate users as necessary, ensuring the system maintains authorized and valid access. This functionality helps in controlling system usage and security.

6. Logout:

Both students and admins can log out securely, terminating the session to prevent unauthorized access and maintain data privacy.

The use cases collectively ensure that the chatbot system provides interactive, secure, and efficient communication, addressing the limitations of traditional information dissemination methods. The combination of user-oriented and admin-oriented functionalities guarantees that the system remains dynamic, reliable, and scalable [15],[16].

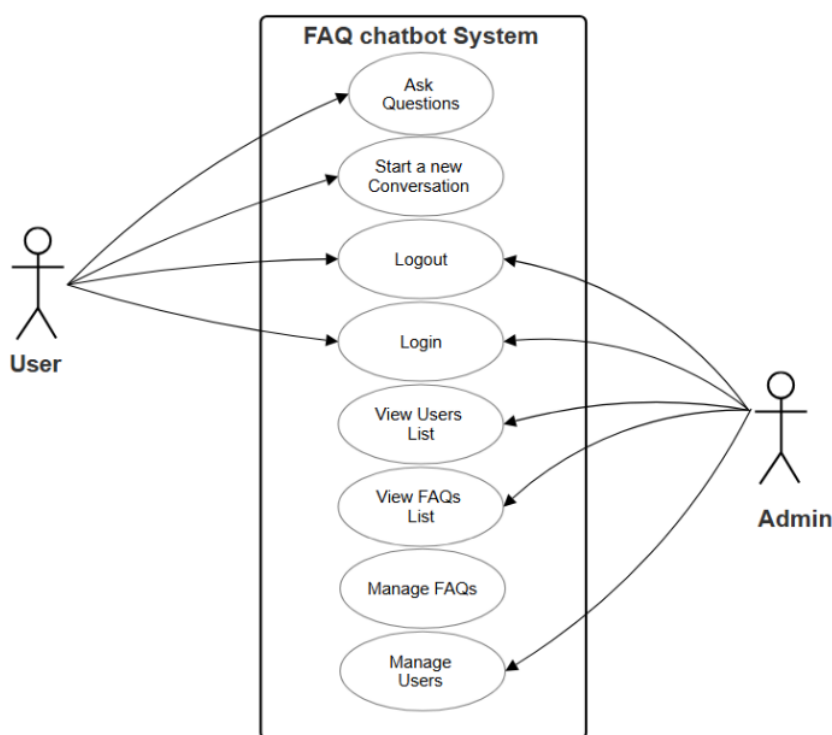


Figure 1: Use case Diagram of the System

3.6. Architecture Design

The Faculty of Computing FAQ Chatbot is designed using a three-tier client-server architecture, which separates the system into three main parts: the client, the application server, and the database server [17]. Users interact with the system through web browsers or mobile devices. These clients only handle the display and user inputs. When a user asks a question or performs any action, the request is sent directly to the application server. The application server, built using the Django REST Framework, is responsible for all the main processing tasks. This includes handling user authentication, searching through FAQ records, performing keyword matching, managing conversation flow, and connecting with external APIs when needed. It acts as the “brain” of the system, making decisions and processing all user requests before sending the appropriate response back to the client. The third layer is the database server. For development, the system uses SQLite for better performance and reliability. This database stores all important information, such as FAQ entries, categories, user accounts, and the history of conversations. The application server retrieves and updates this data whenever required.

Using this layered structure provides several advantages. It improves scalability because each layer can be upgraded or expanded independently. It also increases security by keeping sensitive data on the server rather than on the client side. In addition, the separation of responsibilities makes the system easier to maintain and manage. The React frontend focuses on user interaction, the application server handles logic and processing, and the database ensures that all information is stored securely and consistently [17]. The three-tier architecture is shown in Figure 2.

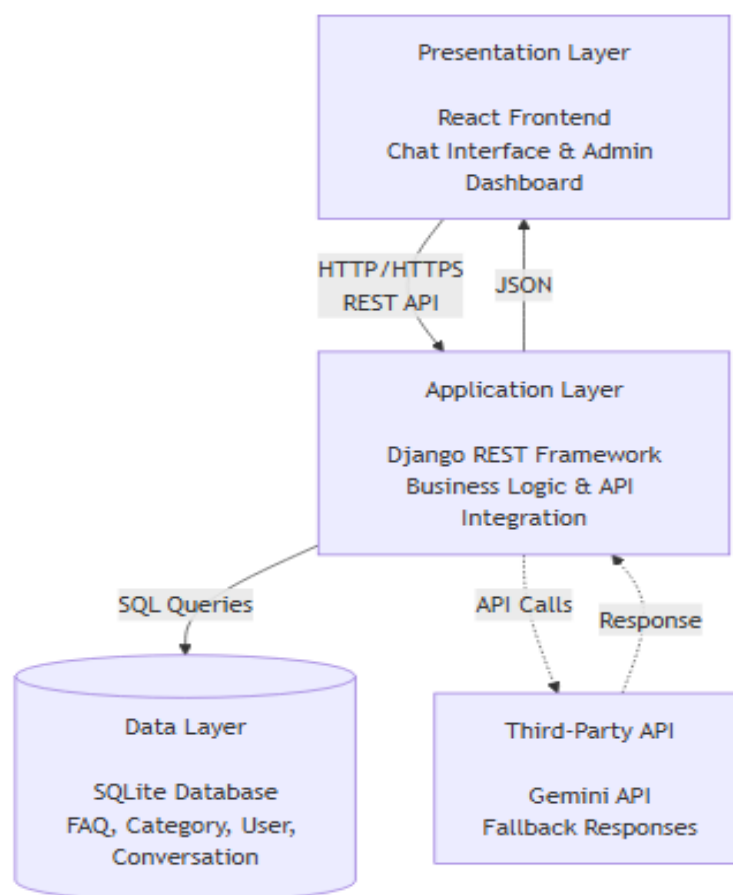


Figure 2: Three-Tier Client-Server Architecture

3.7. Database Design

The database schema is modeled for SQLite. SQLite is chosen for its lightweight nature, zero-configuration requirements, and suitability for prototype development and small to medium-scale applications [18],[19]. The main tables are FAQ, User, and Conversation. Each user can have one or more conversation records (One-to-many on UserID). FAQ stores question text, answer content, and keywords used by the matching algorithm. User represents both regular users (students/staff) and administrators who manage the system. Conversation records each interaction event, linking a user to an FAQ (if matched) and noting the query, response, timestamp, and response source (FAQ database or third-party API). Foreign keys enforce referential integrity (e.g., Conversation). UserID references User. UserID, and Conversation FAQID references FAQ.FAQID. This design ensures that FAQ data is normalized and that user interactions can be tracked for analytics and system improvement [18],[19].

Overall, the architecture and design provide a clear blueprint for building the Faculty of Computing FAQ Chatbot system: a React-based frontend (client) connecting to a secure Django REST Framework application

server, which in turn manages an SQLite database with the above schema. All sensitive operations (e.g., adding FAQs, user authentication, and query processing) are handled on the server side to ensure data integrity and security [18]. The key entities and relationships are shown in Figure 3.

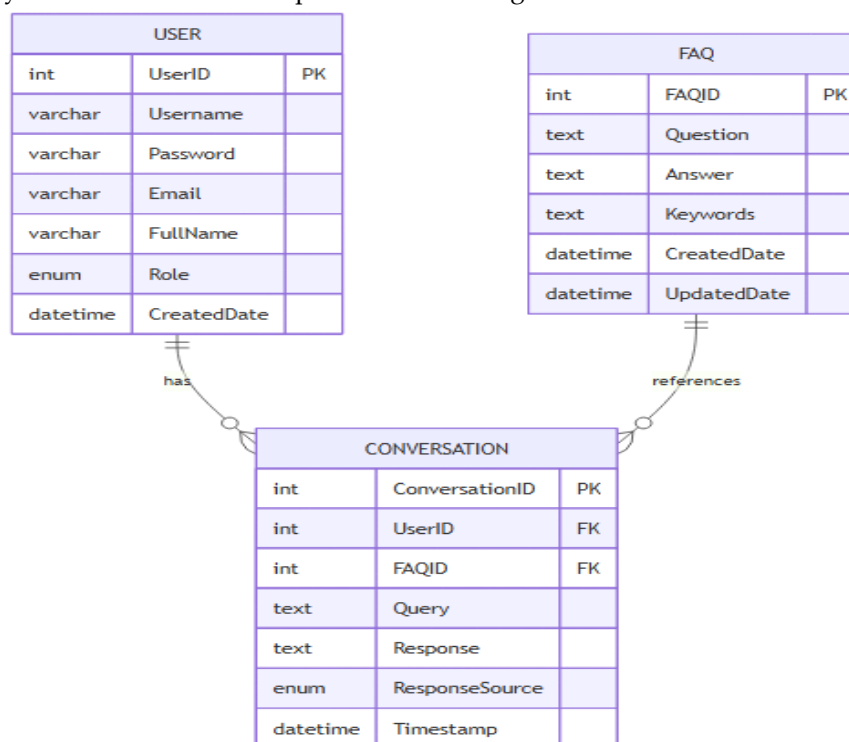


Figure 3: Entity Relationship Diagram

4. Implementation

The development of the FAQ Chatbot for the Faculty of Computing involved several tools and technologies [21]. JavaScript and Python were the primary programming languages, with JavaScript used for the frontend interface and Python handling backend processing and database management. The frontend was implemented using React, providing an interactive and responsive user experience with fast page rendering and reusable components. On the backend, Django REST Framework was employed to create APIs that handle user questions and return responses from the database. The backend was hosted on Render, allowing automatic deployment and scalability. For database management, SQLite was chosen to store FAQ questions and answers, with the initial data prepared in JSON format and subsequently imported into the database. Visual Studio Code served as the development environment, offering debugging tools, extensions, and support for both Python and JavaScript. Git and GitHub were used for version control, enabling tracking of code changes and collaborative development. Additionally, Postman was used to test the APIs, ensuring correct responses were returned during development.

The chatbot operates using a search-based question-matching approach. Instead of generating answers, it retrieves the closest matching FAQ from the database and delivers it to the user. The process begins with data preparation, where FAQ records are created in JSON format and imported into SQLite, with each record containing a question, answer, category, and creation date. On the backend, user questions submitted through the frontend are converted to lowercase and matched against stored FAQs. If a relevant FAQ is found, the corresponding answer is returned; otherwise, a fallback message prompts the user to contact the admin. The system also supports admin management, allowing administrators to log in via a React dashboard to add, edit, or delete FAQ entries and remove users. All changes are immediately reflected in the chatbot. Users can create

accounts, maintain personal chat histories, and ask multiple questions at any time, ensuring seamless and continuous access to faculty information.

4.1. Description of the Developed System

Register page: Figure 4 shows the interface a new user uses to enter his/her details before signing up for the system.

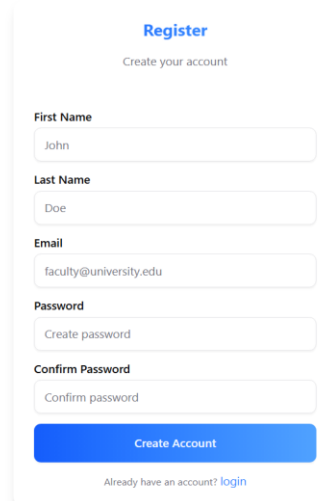
The image shows a 'Register' form with the title 'Register' and subtitle 'Create your account'. It contains input fields for 'First Name' (with 'John' entered), 'Last Name' (with 'Doe' entered), 'Email' (with 'faculty@university.edu' entered), 'Password' (with 'Create password' entered), and 'Confirm Password' (with 'Confirm password' entered). A blue 'Create Account' button is at the bottom, with a link 'Already have an account? login' below it.

Figure 4: The Register Page

Login Page: Figure 5 illustrates the interface where both administrators and regular users can log into the system. After authentication, each user is redirected to their authorized dashboard based on their role.

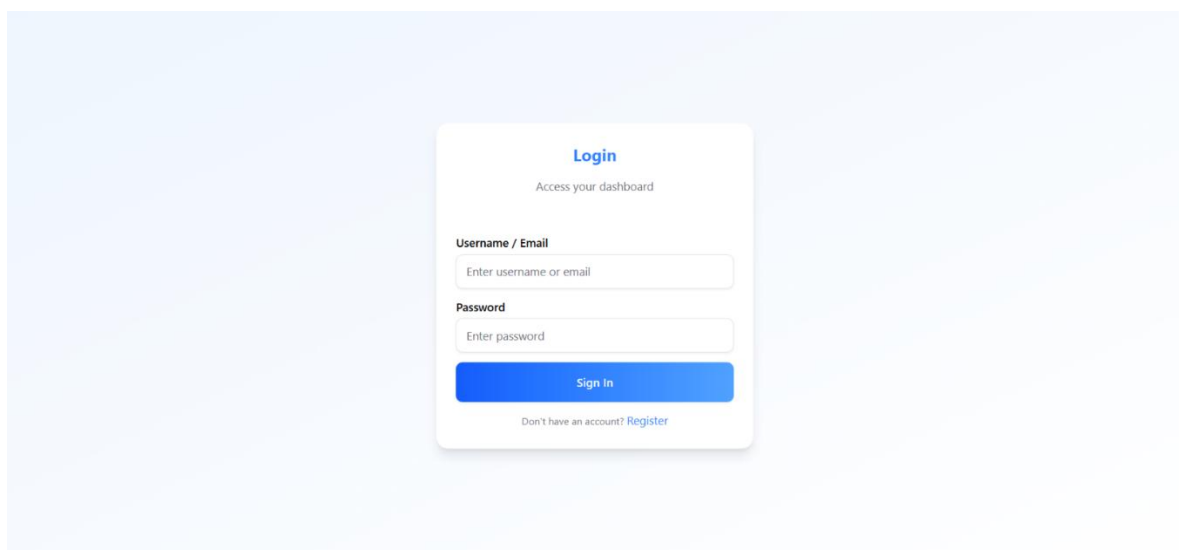
The image shows a 'Login' form with the title 'Login' and subtitle 'Access your dashboard'. It contains input fields for 'Username / Email' (with 'Enter username or email' entered) and 'Password' (with 'Enter password' entered). A blue 'Sign in' button is at the bottom, with a link 'Don't have an account? Register' below it.

Figure 5: The Login Page

Chatbot Interface: Figure 6 displays the chatbot interface, where users can input their questions and receive responses. It also displays the chat history for returning users, while remaining empty for new users.

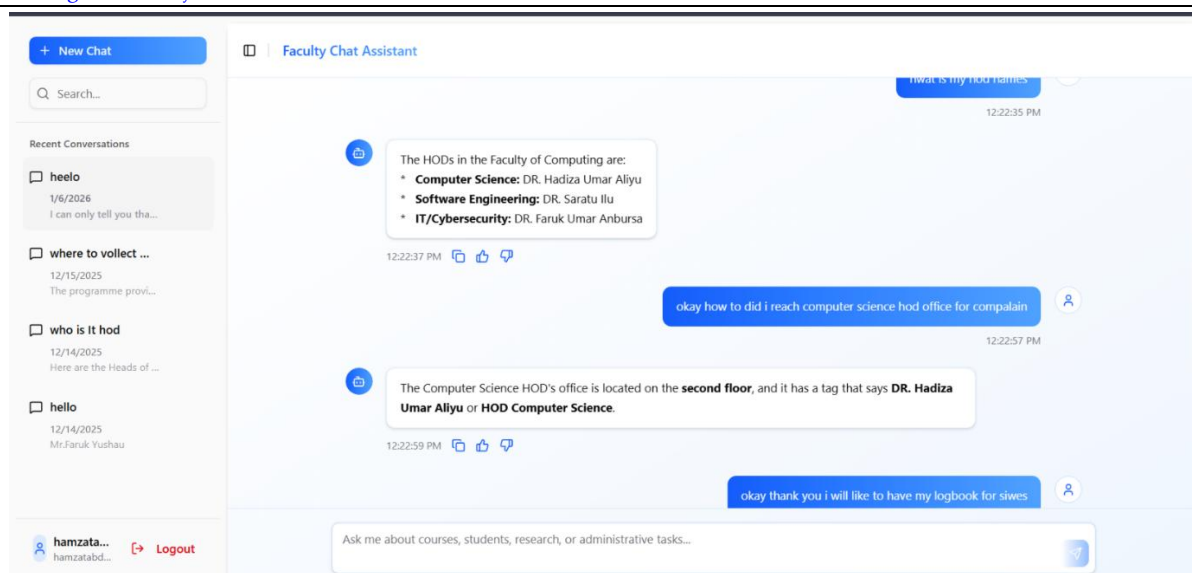


Figure 6: The Chatbot Interface

Admin Dashboard: Figure 7 shows the administrator's dashboard, which displays the list of queries as well as the users registered in the system.

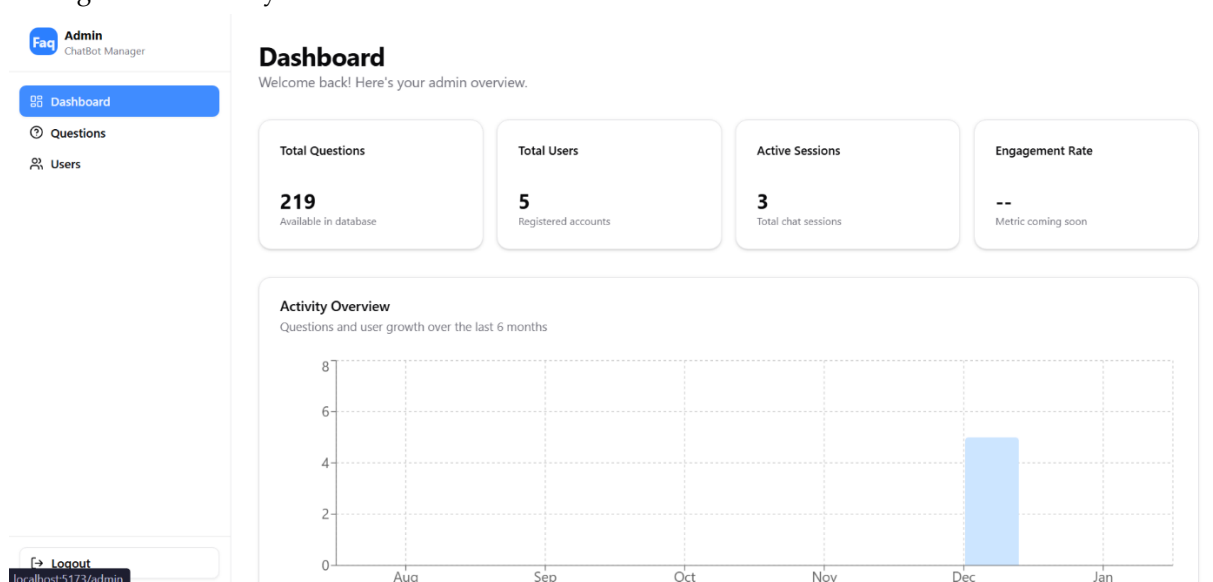


Figure 7: Admin Dashboard

Admin Delete Question: Figure 8 displays the interface where administrators can delete questions from the system. This functionality ensures that the question database remains accurate, up-to-date, and aligned with the system's requirements.

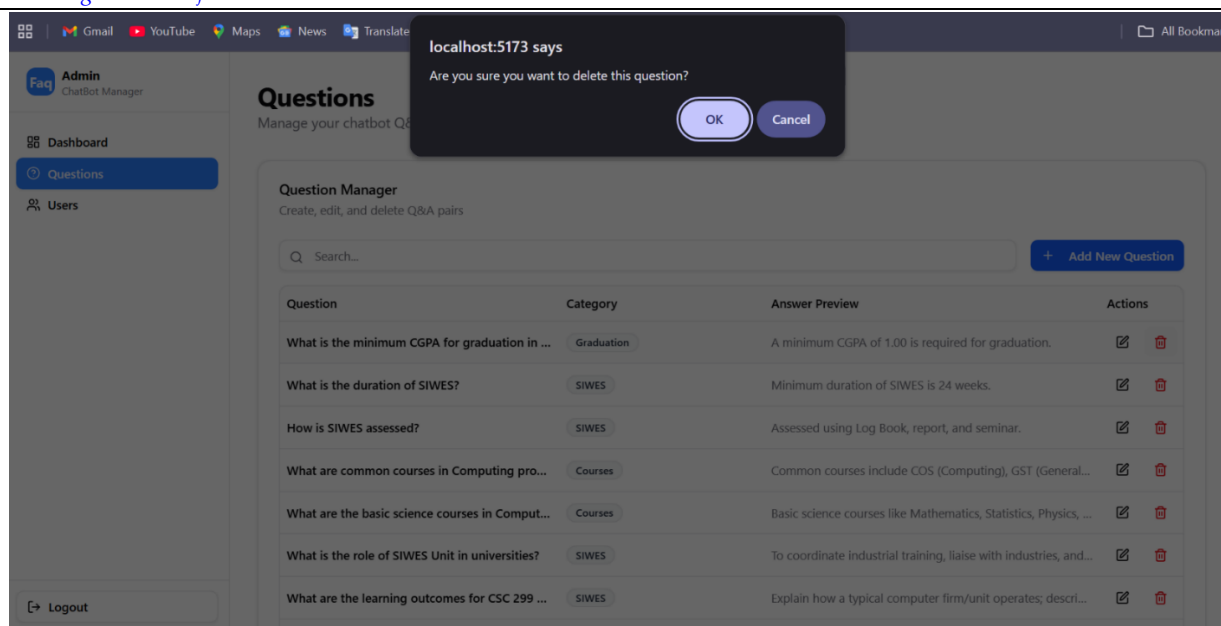


Figure 8: Admin Delete Question

Admin Add Question: Figure 9 displays the interface where administrators can input new questions into the system. This functionality allows for the continuous expansion of the question bank, ensuring comprehensive coverage of topics and alignment with the system's evolving requirements.

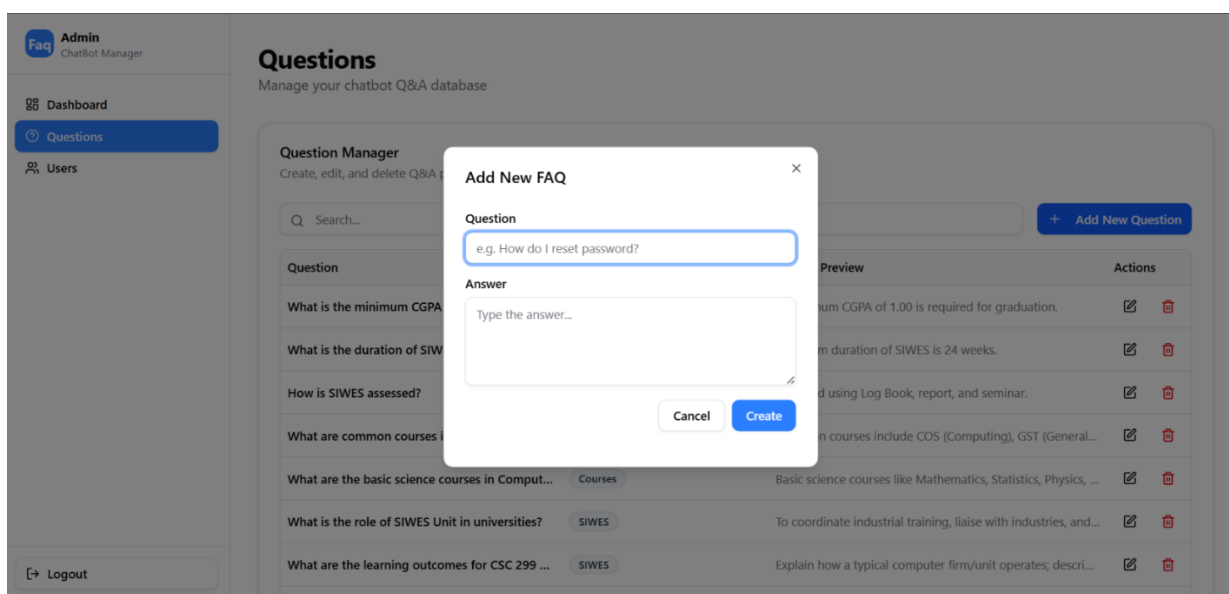


Figure 9: Admin Add Question

Admin Edit Question: Figure 10 illustrates the interface that enables administrators to modify existing questions. This functionality is crucial for correcting errors, refining phrasing, or updating content, ensuring the question database remains accurate and reliable over time.

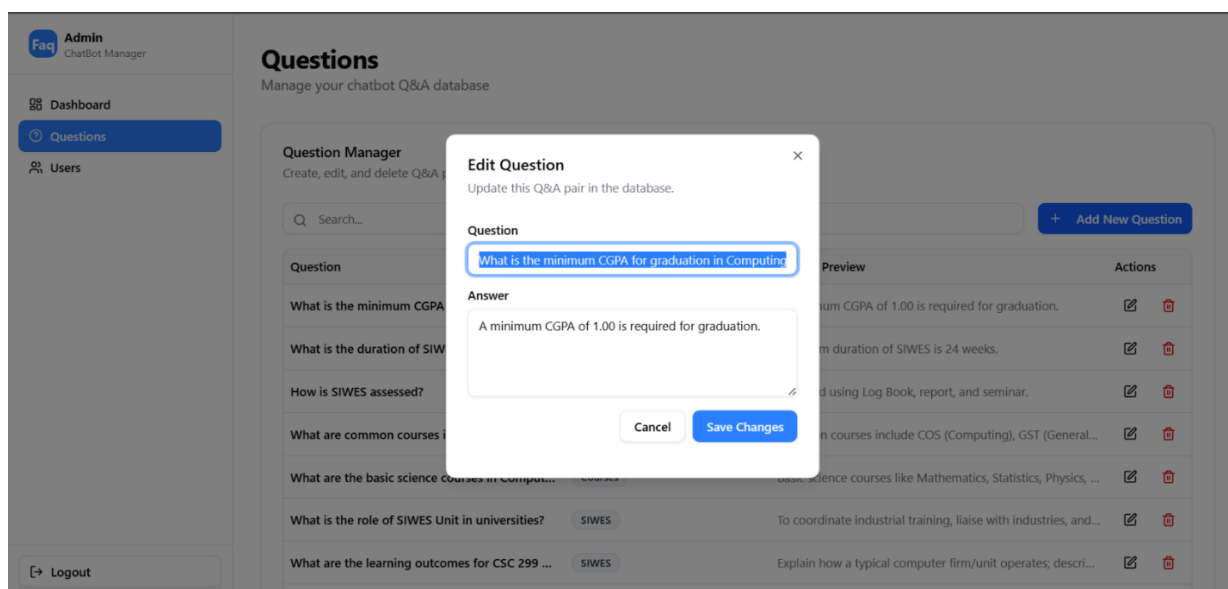


Figure 10: Admin Edit Question

Admin Question Interface: Figure 11 shows the interface that displays all the questions in the system. Administrators can edit any question when it becomes outdated or when changes are required

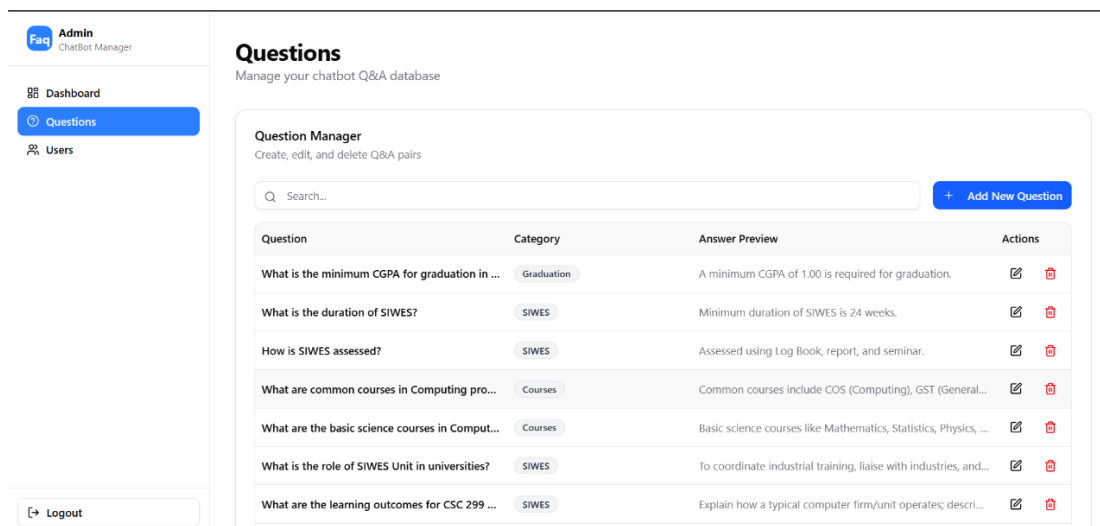


Figure 11: Admin Question Interface

5. Testing

The chatbot was validated against its requirements using unit and integration testing, ensuring that each component functioned correctly both independently and when combined within the system. Unit testing focused on individual modules such as the question-matching logic and database API endpoints, while integration testing verified the seamless interaction between the frontend, backend, and database [20], [21]. These testing practices follow established software engineering methodologies for system validation, ensuring reliability, accuracy, and maintainability of the deployed chatbot [21].

5.1. Unit Testing

Table 1 shows the unit testing process aimed at validating the individual components of the FAQ chatbot. To ensure system reliability and response accuracy, we tested user authentication, the FAQ search algorithm,

exception handling, and administrative management functions. All components were verified to function as expected.

Table 1: Unit Testing Table

S/N	Component	Test Description	Result
1	User Registration	Check if new users can create accounts successfully.	Passed
2	User Login	Verify that only valid users can log in.	Passed
3	FAQ Search	Confirm system returns the closest related answers.	Passed
4	No Answer Handling	Check if the “no answer found” message displays correctly.	passed
5	Admin Edit FAQ	Verify that updates to FAQs reflect immediately.	Passed
6	Admin Add FAQ	Ensure the admin can create new FAQs successfully.	Passed

5.2. Integration Testing

Integration testing ensured that all system components communicate correctly, verifying the seamless data flow between the user interface, the backend server, and the database, as shown in Table 2.

Table 2: Integration Testing Table

S/N	Integration	Test Description	Result
1	Frontend to Backend	Verify React sends requests and receives responses.	Passed
2	Backend to Database	Check that FAQs are read and stored correctly.	Passed
3	Admin Dashboard to API	Ensure added questions appear in the chatbot.	Passed
4	Auth to Chat Interface	Verify logged-in users can successfully use the chat interface to send messages.	passed

6. Conclusions

The FAQ Chatbot developed for the Faculty of Computing at Bayero University Kano has effectively addressed the challenges associated with accessing timely and accurate academic information. Students and staff often encounter difficulties obtaining details about course registration, level coordinators, results processing, faculty policies, and general inquiries. By automating responses to these frequently asked questions, the chatbot reduces the workload of administrative staff and ensures that information is readily accessible at any time.

The system was developed following standard software engineering processes, including requirement analysis, system design, implementation, and testing. The backend was implemented using Django REST Framework, while the frontend was built with React and TypeScript, providing a modern, interactive, and user-friendly interface. FAQ data was initially stored in JSON format within SQLite, with provisions for future migration to PostgreSQL. Users can create accounts, log in, and interact with the chatbot via a chat interface. Questions are matched against the FAQ database, and relevant answers are returned instantly. Unmatched queries generate a fallback message, allowing administrators to update the database via the admin dashboard.

Deployment of the system utilized Render for the backend and Vercel for the frontend, and testing confirmed that all core functionalities including login, chatbot response, admin updates, and backend-frontend communication are performed correctly. The project demonstrates that chatbots are effective tools in academic environments, capable of improving communication, streamlining access to information, and reducing

administrative pressure. The system is flexible, scalable, and suitable for future enhancements.

Future improvements may include integrating more advanced natural language processing models to handle complex and nuanced queries, migrating to a full PostgreSQL database for enhanced performance and security, and providing administrators with an analytics dashboard to monitor frequently asked questions, unanswered queries, and user activity. Additionally, the system can be extended to support multiple faculties, developed as a mobile application to increase accessibility, and enhanced with a user feedback mechanism to maintain up-to-date and accurate responses. These enhancements will strengthen the chatbot's overall functionality, scalability, and contribution to efficient academic communication within the university.

Author contributions: All authors must specify their roles in the research. Use structured contributions such as: Conceptualization, H.A.K., and H.A.; Methodology, H.A., and H.A.K.; Writing—original draft, H.A.; Writing—review and editing, Z.M.M. Only contributors with substantial involvement should be listed as authors.

Funding Statement: This research received no external funding.

Acknowledgments: The authors would like to thank the editorial board and reviewers for their insightful comments and suggestions, which helped to improve the quality of this manuscript.

Conflict of interest: The authors declare no conflict of interest.

References

- [1] B. Bockelmann, J. Smith, and A. Kumar, "Access to information and efficient communication in academic institutions," *Journal of Higher Education Studies*, vol. 12, no. 3, pp. 45–58, 2025.
- [2] A. Iordache-Platis and A. Josan, "Challenges in university information dissemination: Traditional methods vs. digital approaches," *International Journal of Educational Technology*, vol. 5, no. 2, pp. 21–33, 2009.
- [3] R. Aggarwal, L. Chen, and P. Singh, "Automated systems for improving accessibility and communication in higher education," *Education and Information Technologies*, vol. 29, pp. 1125–1140, 2024.
- [4] S. Ahmad, T. Rahman, and F. Malik, "Designing intelligent conversational chatbots for university FAQ support," *Journal of Applied Computing and Informatics*, vol. 20, no. 1, pp. 77–90, 2025.
- [5] N. Dwivedi, M. Patel, and R. Joshi, "Integrating web technologies and NLP in academic support systems," *International Journal of Computer Applications*, vol. 182, no. 9, pp. 1–12, 2024.
- [6] P. Ranoliya, R. C. Bhatt, and M. T. Zaveri, "AIML-based hybrid chatbot system for automating university FAQs," *Procedia Computer Science*, vol. 115, pp. 200–207, 2017.
- [7] F. Clarizia, G. Lax, and P. S. Rizzuti, "An ontology-based virtual assistant for educational support," *Computers & Education*, vol. 122, pp. 123–135, 2018.
- [8] J. Lee, H. Kim, and S. Park, "Evaluating FAQ chatbots to reduce administrative workload in universities," *International Journal of Human-Computer Studies*, vol. 127, pp. 25–37, 2019.
- [9] R. Huddar, M. Patel, and K. Shah, "Dexter: A college FAQ chatbot using NLP for intent recognition," *Education and Information Technologies*, vol. 25, pp. 1543–1558, 2020.
- [10] S. Han and H. Lee, "FAQ chatbots and inclusive learning in MOOCs: An experimental study," *Computers in Human Behavior*, vol. 133, 107306, 2022.
- [11] G. Labadze, D. Silva, and P. Kumar, "A systematic review of AI chatbots in higher education: Impacts and challenges," *International Journal of Educational Technology in Higher Education*, vol. 20, no. 1, pp. 1–27, 2023.

- [12] H. Bilquise, A. Almarzouqi, and F. Hassan, "Student acceptance of academic advising chatbots using TAM and UTAUT," *Computers & Education*, vol. 187, 104623, 2024.
- [13] P. Mangotra, V. Singh, and R. Arora, "Deep learning-based chatbot for student support in higher education," *Journal of Educational Computing Research*, vol. 62, no. 4, pp. 789–805, 2024.
- [14] M. Zaky and D. Ratnasari, "Dual-mode academic chatbot: Rule-based and hybrid reasoning approach," *Journal of Intelligent Systems*, vol. 34, no. 2, pp. 401–416, 2025.
- [15] "Agile development methodology: Iterative and incremental development cycles enhance software delivery by supporting flexibility and continuous validation," *International Journal of Advances in Science, Engineering and Technology*, vol. 2, no. 2, pp. 1–6, Apr. 2014. [Online]. Available: https://www.ijar.in/journal/journal_file/journal_pdf/6-46-139763398055-59.pdf
- [16] M. T. Akbar and M. U. Siregar, "A survey on software requirements engineering in information technology institutions," *Elinvo (Electronics, Informatics, and Vocational Education)*, vol. 9, no. 2, pp. 93–104, 2023, doi: 10.21831/elinvo.v9i2.70862.
- [17] M. L. Young, *The Architecture of Modern Web Applications: Three-Tier Client–Server Systems*, 2nd ed., New York, NY, USA: Springer, 2019.
- [18] R. J. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Boston, MA, USA: Pearson, 2016.
- [19] S. Hipp, "SQLite: Documentation and Architecture Overview," SQLite Documentation, 2020. [Online]. Available: <https://www.sqlite.org/docs.html>
- [20] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2020.
- [21] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015.



Disclaimer/Publisher's Note: The views, opinions, and content expressed in all articles are solely those of the respective author(s) and contributor(s) and do not necessarily reflect those of the JSSCI, its editors, or the publisher. JSSCI and its editorial team assume no responsibility for any harm or damage resulting from the use of information, methods, or products mentioned in the publication.